WAITING ROOM

A clinic appointment system, shipped through a complete DevOps pipeline



Mrinmay Dev Sarma

DevOps · Instructor: Nensi Ravaliya

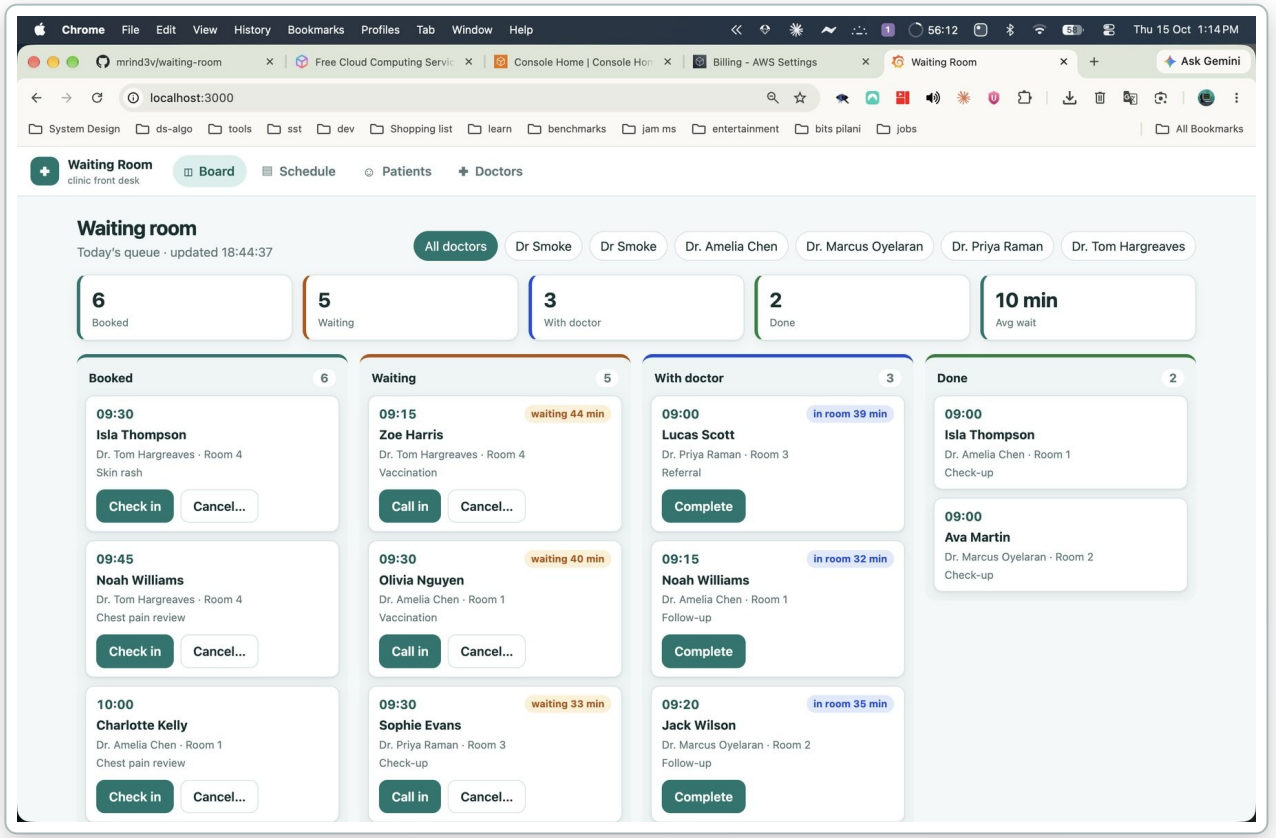
One desk. One live queue. Zero double bookings

Books appointments by doctor and time slot

Checks in patients the moment they arrive

Shows who waits, who sees a doctor, and for how long

Blocks double bookings inside the database



The live queue board

The project in numbers

16

REST endpoints, plus health, ready and metrics

36

tests pass at 97% coverage

6

CI/CD jobs run on every push

50

AWS resources: created and destroyed by Terraform

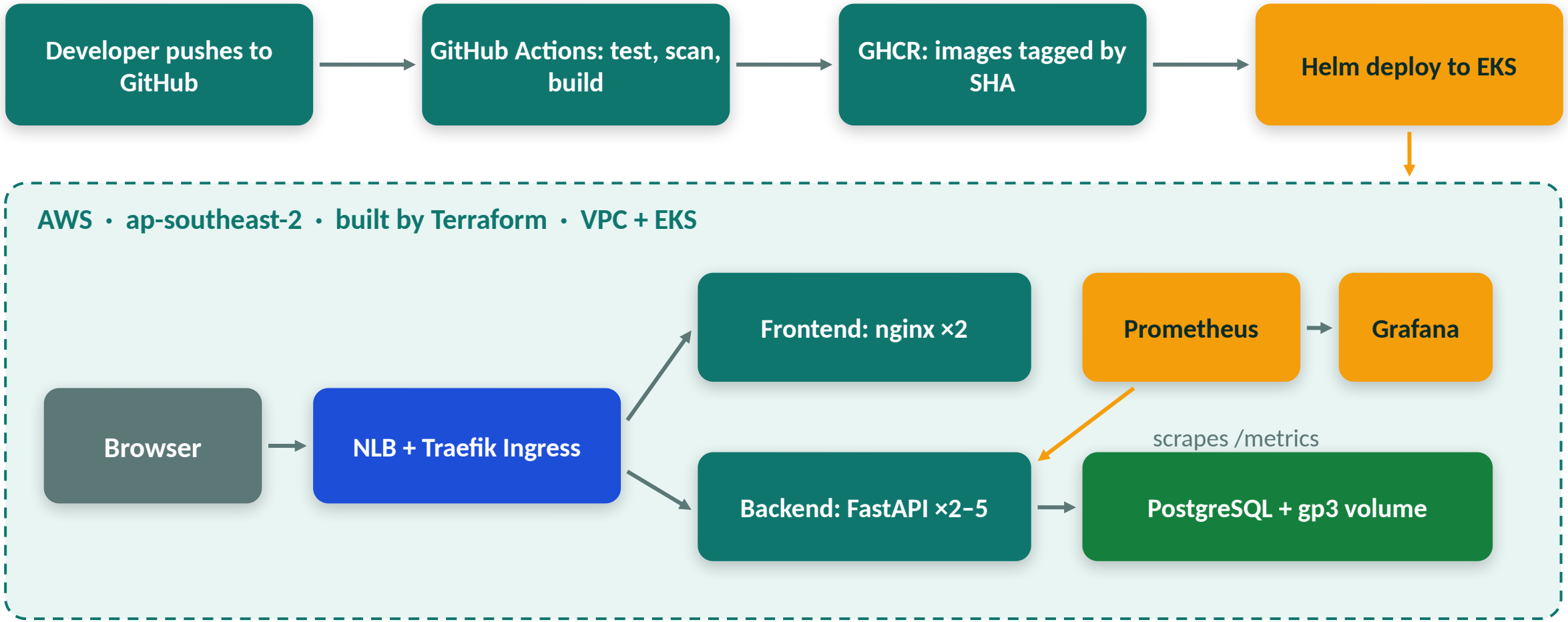
$0.26

per hour for the whole AWS stack

0

secrets committed to Git

How it all fits together



01

The application

FastAPI, React and PostgreSQL, built around one hard rule: never double-book a doctor

Four screens run the front desk

Board

Four live columns with waiting timers and KPIs

Schedule

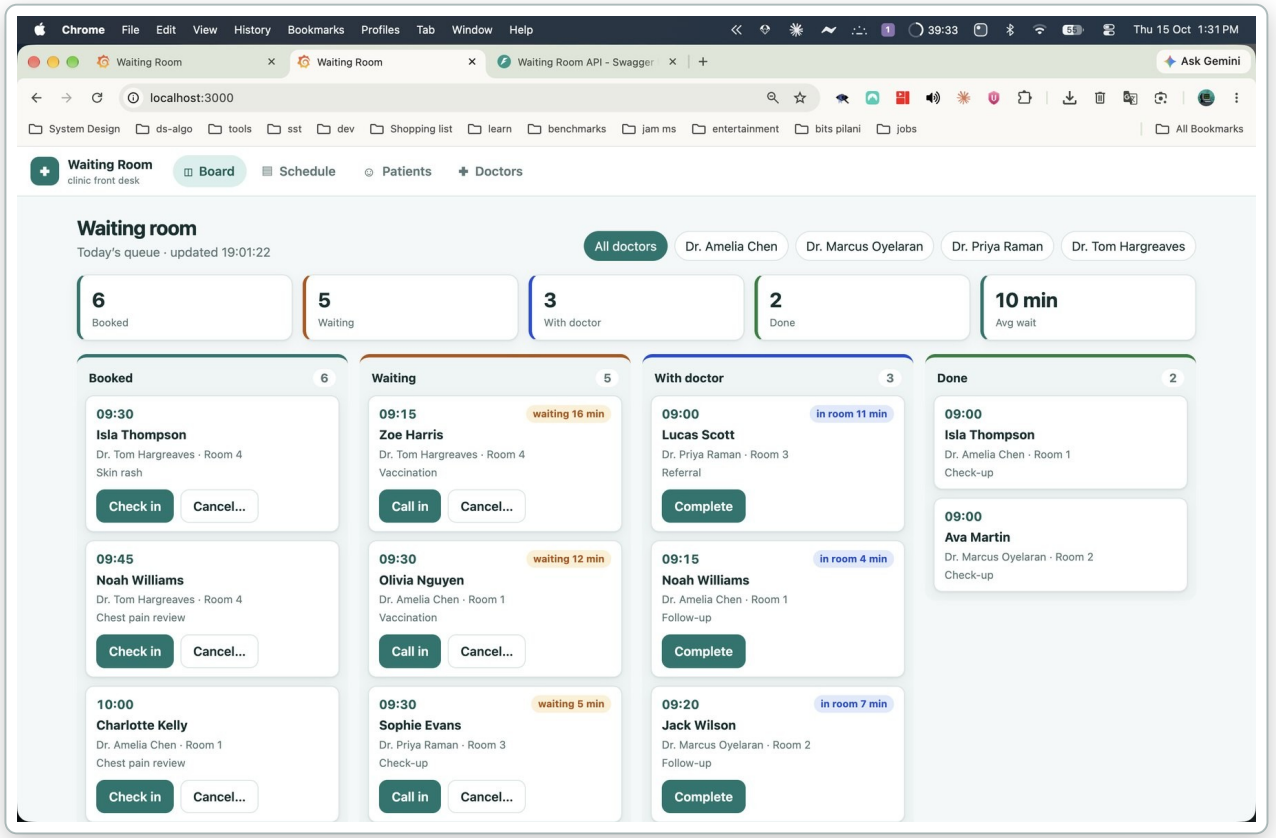
Each doctor's day. Click a free slot to book

Patients

Search, register, edit and delete

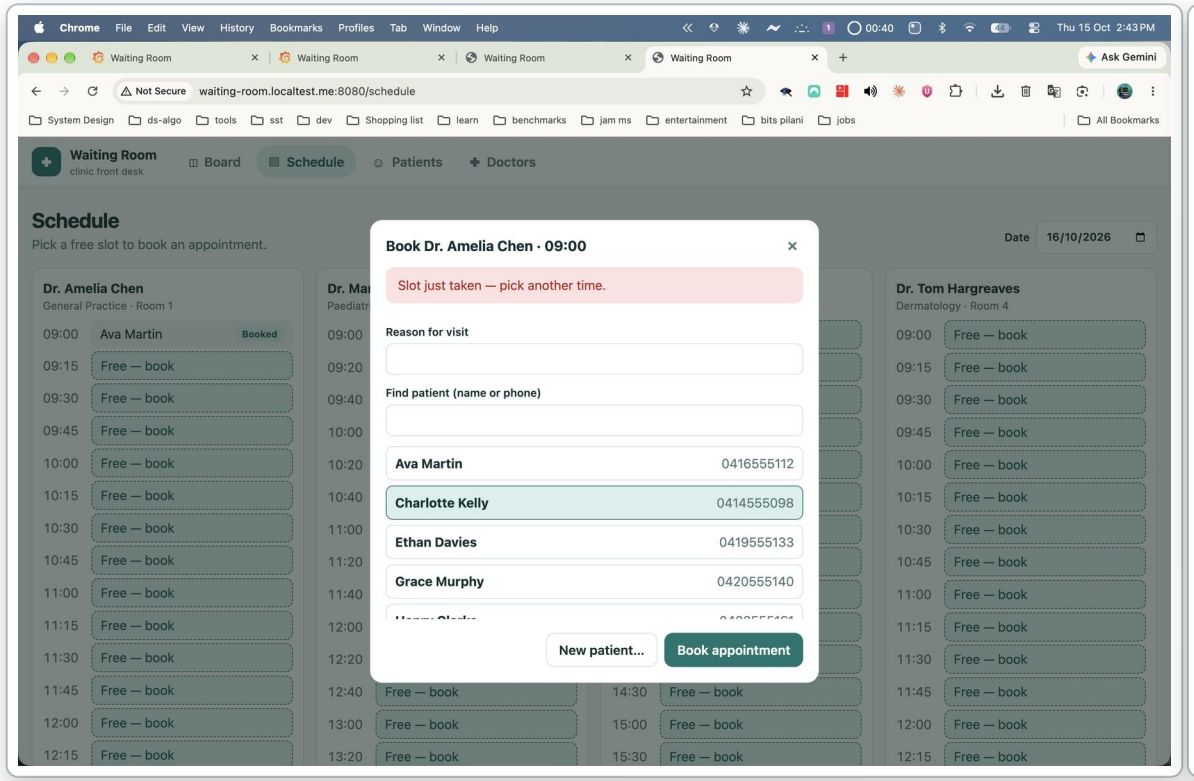
Doctors

Specialty, room, hours and slot length

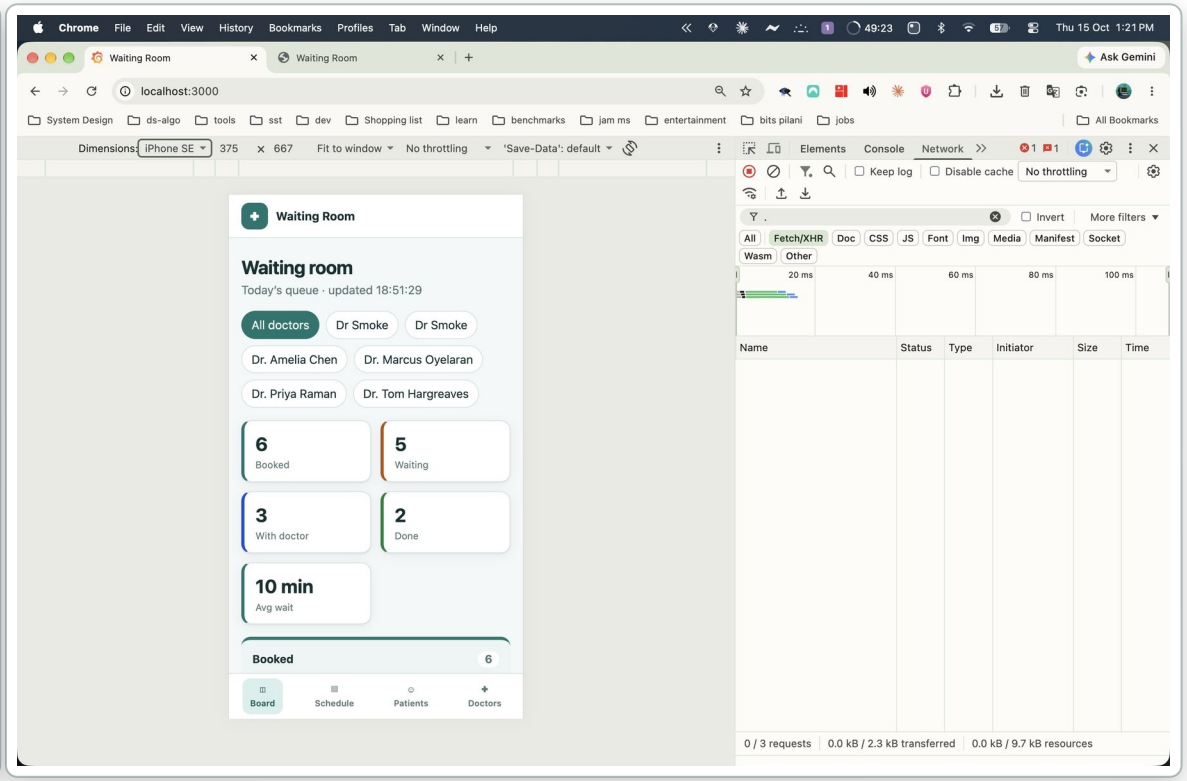


The Board, served from Docker Compose

Users see conflicts instantly, on any screen size

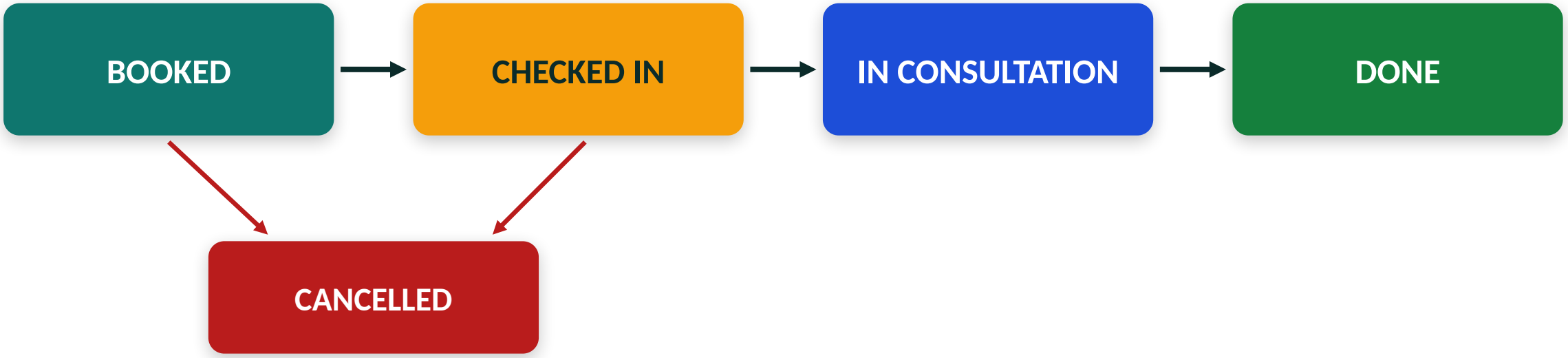


A taken slot shows an inline error, not a crash



The layout adapts down to 375 px

Every appointment follows one lifecycle



Illegal moves return 409 and list the allowed next steps

Each move stamps a time, so the board shows live waiting

Cancelling frees the slot for the next patient

PostgreSQL blocks double bookings, not the app

```
EXCLUDE USING gist (
  doctor_id WITH =,
  tstzrange(starts_at, ends_at, '[)')
    WITH &&
) WHERE (status <> 'CANCELLED')
```

Atomic: the database decides, so two clicks in the same millisecond cannot both win

Precise: partial overlaps fail, back-to-back slots pass

Self-healing: cancelling frees the slot at once

The proof: a concurrency test fires two bookings at once. One request returns 201 and the other returns 409 SLOT_TAKEN.

Sixteen endpoints plus three ops probes

Doctors

list · create · update · availability

Patients

search · detail · create · update · delete

Appointments

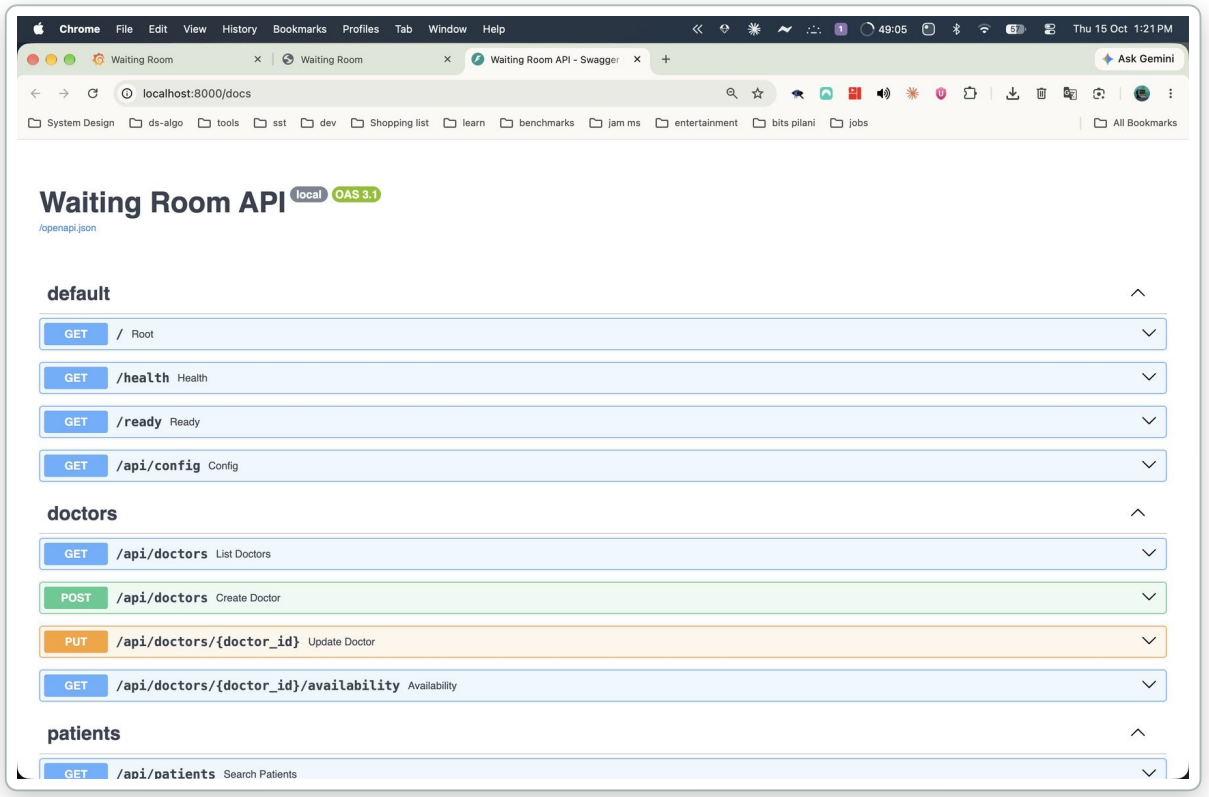
list · detail · book · reschedule · status · delete

Dashboard

today: counts, average wait, next up

Ops

/health · /ready · /metrics



Swagger documents every route at /docs

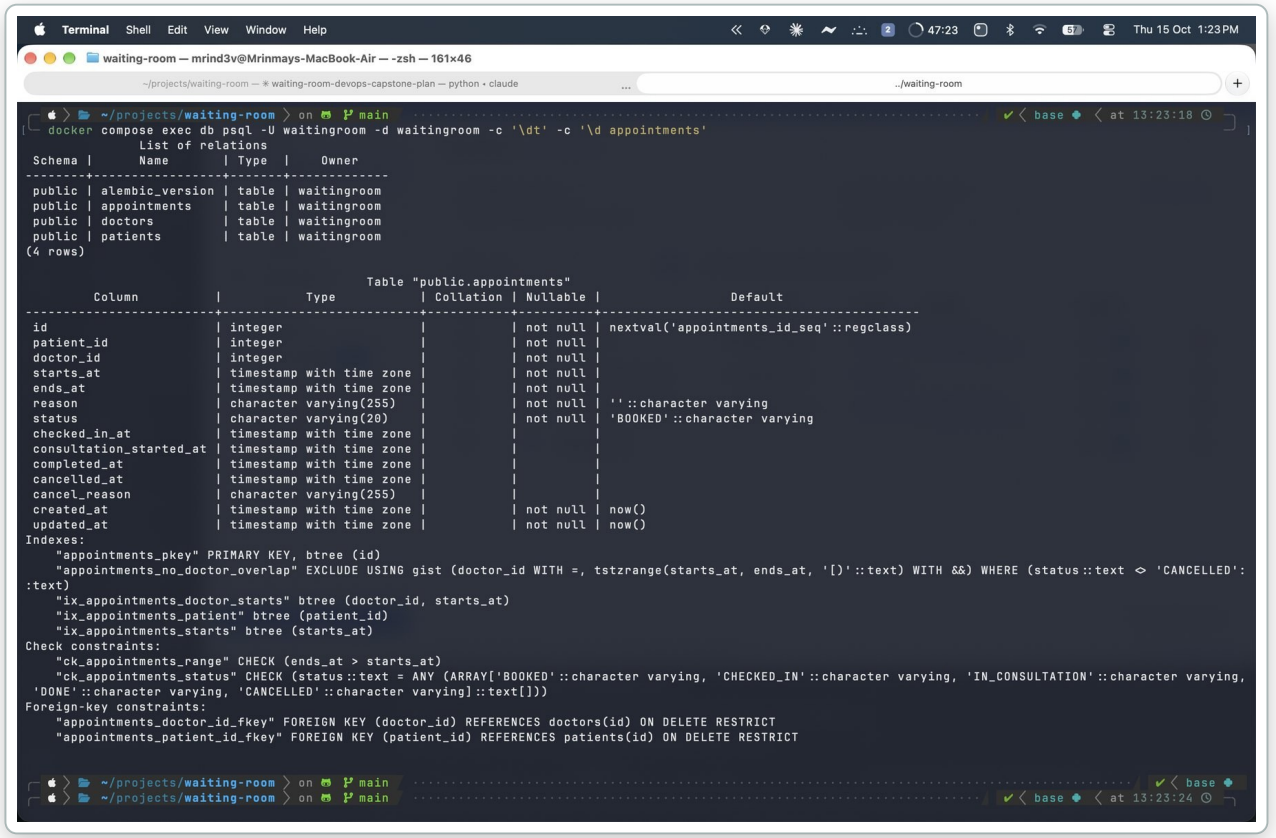
Alembic owns the schema

Two migrations create doctors, patients and appointments

Migrations only: the app never calls create_all

Advisory lock: concurrent pods migrate one at a time

Readiness: /ready returns 503 until the schema reaches head



Tables and the exclusion constraint in psql

02

Quality and containers

Tests prove the rules. Docker makes the app run the same everywhere

36 tests prove the rules

36

tests pass

97%

code coverage

Real PostgreSQL: the test database enforces the constraint

Frozen clock: time rules never flake

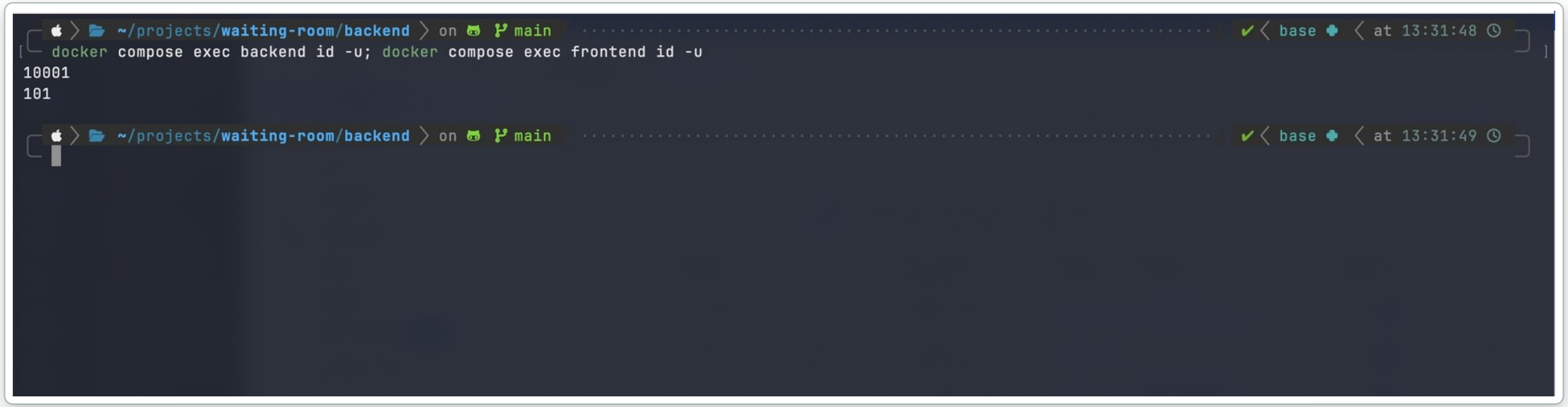
Race test: two threads, one winner

```
============================================================================ tests coverage ============================================================================
_____________________________________________________ coverage: platform darwin, python 3.12.13-final-0 _____________________________________________________

Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
app/__init__.py                   0      0   100%
app/config.py                    26      1    96%   27
app/db.py                        13      0   100%
app/deps.py                      14      0   100%
app/errors.py                    23      0   100%
app/main.py                      24      3    88%   16-17, 25
app/metrics.py                    4      0   100%
app/models.py                    46      0   100%
app/routers/__init__.py           0      0   100%
app/routers/appointments.py     110      3    97%   85, 104, 159
app/routers/dashboard.py         20      1    95%   25
app/routers/doctors.py           52      0   100%
app/routers/ops.py               37      2    95%   45-46
app/routers/patients.py          68      6    91%   66, 78-84
app/schemas.py                  114      1    99%   86
app/seed.py                      42      1    98%   116
app/status_flow.py               18      0   100%
-----------------------------------------------------------
TOTAL                           611     18    97%
Required test coverage of 85% reached. Total coverage: 97.05%
============================================================================ 36 passed in 1.19s ============================================================================
```

pytest -v with the coverage report

Docker images run as non-root



Multi-stage builds

Node builds. nginx serves. No build tools ship

Digest-pinned bases

Every build uses the same image bytes

Non-root users

uid 10001 for the API, uid 101 for nginx

One command starts the whole stack

```
backend-1   | INFO:     127.0.0.1:35712 - "GET /ready HTTP/1.1" 200 OK

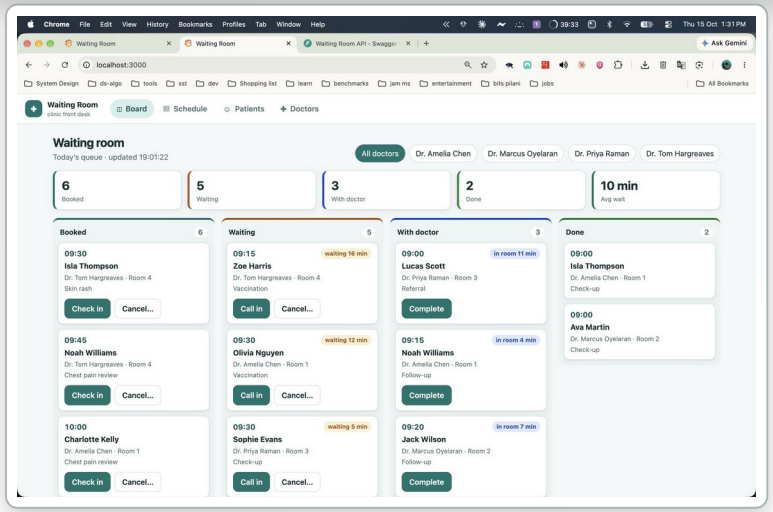
~/projects/waiting-room/backend on main                    took 2m 18s  base  at 13:27:37
docker compose ps
NAME                      IMAGE                                                                                  COMMAND                  SERVICE    CREATE
D         STATUS                    PORTS
waiting-room-backend-1    waiting-room-backend                                                                   "uvicorn app.main:ap…"   backend    2 minu
tes ago   Up 2 minutes (healthy)    0.0.0.0:8000→8000/tcp, [::]:8000→8000/tcp
waiting-room-db-1         postgres:17-alpine@sha256:b0f9560a2de083e2cc7382e75f808c7381a32852a7ec49117deedb300e552b24   "docker-entrypoint.s…"   db         2 minu
tes ago   Up 2 minutes (healthy)    127.0.0.1:5432→5432/tcp
waiting-room-frontend-1   waiting-room-frontend                                                                  "/docker-entrypoint.…"   frontend   2 minu
tes ago   Up 2 minutes              0.0.0.0:3000→8080/tcp, [::]:3000→8080/tcp

~/projects/waiting-room/backend on main                                                  base  at 13:27:40
```

db → migrate → backend → frontend: each step waits for the previous one to be healthy

Secrets come from an ignored .env file, never from Git

Postgres listens on localhost only

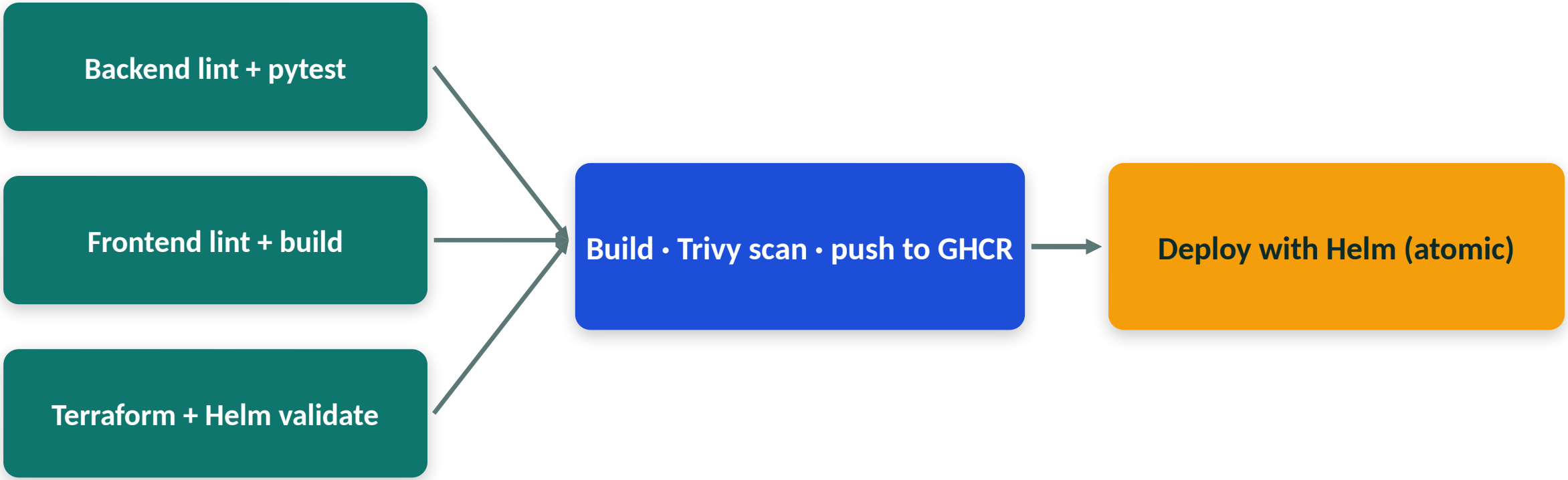


03

The pipeline

Every push to main is tested, scanned, built, published and deployed

Every push runs six jobs

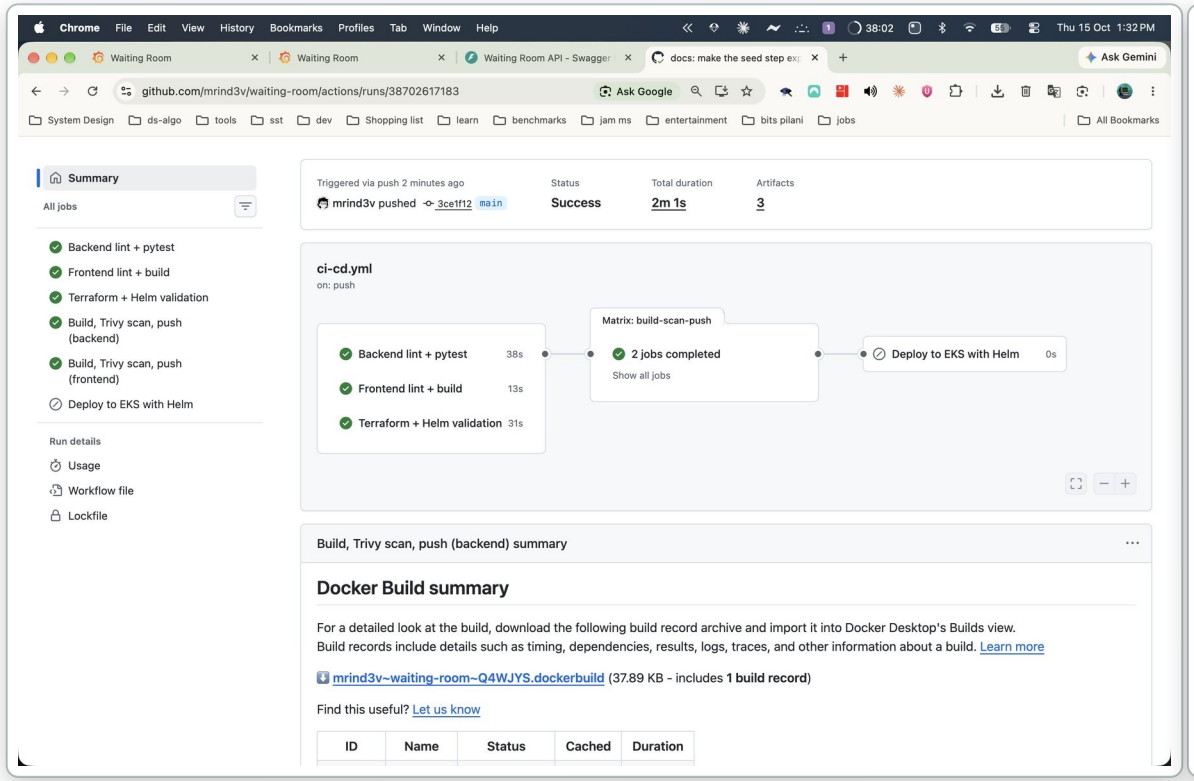


On push and PR to main

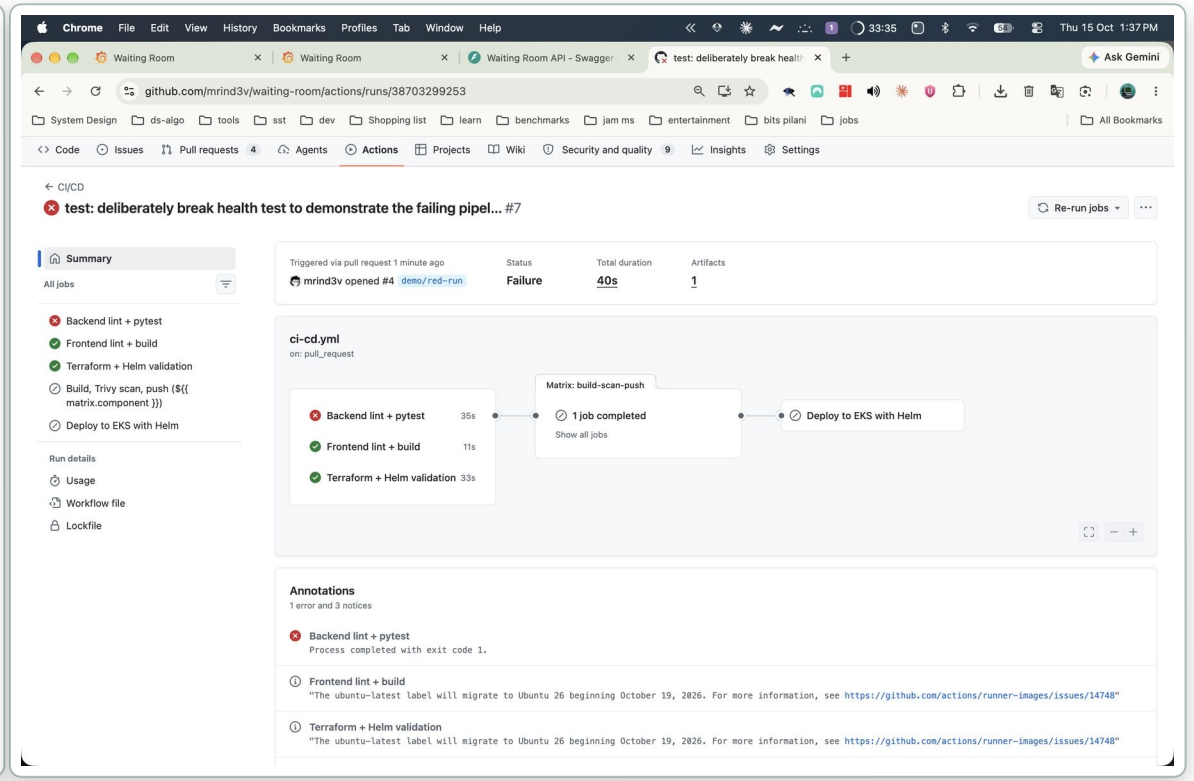
Tag = full Git SHA, never latest

Deploy only when enabled

A failing test stops everything

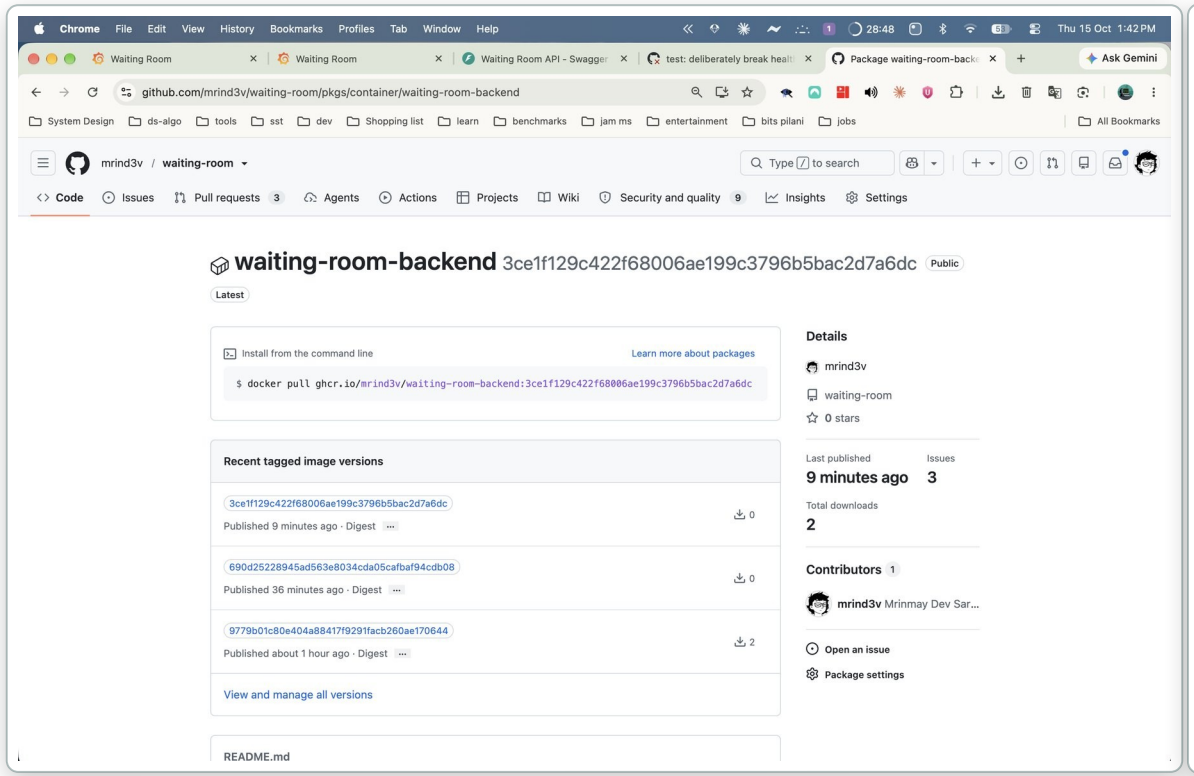


Green: every job passes

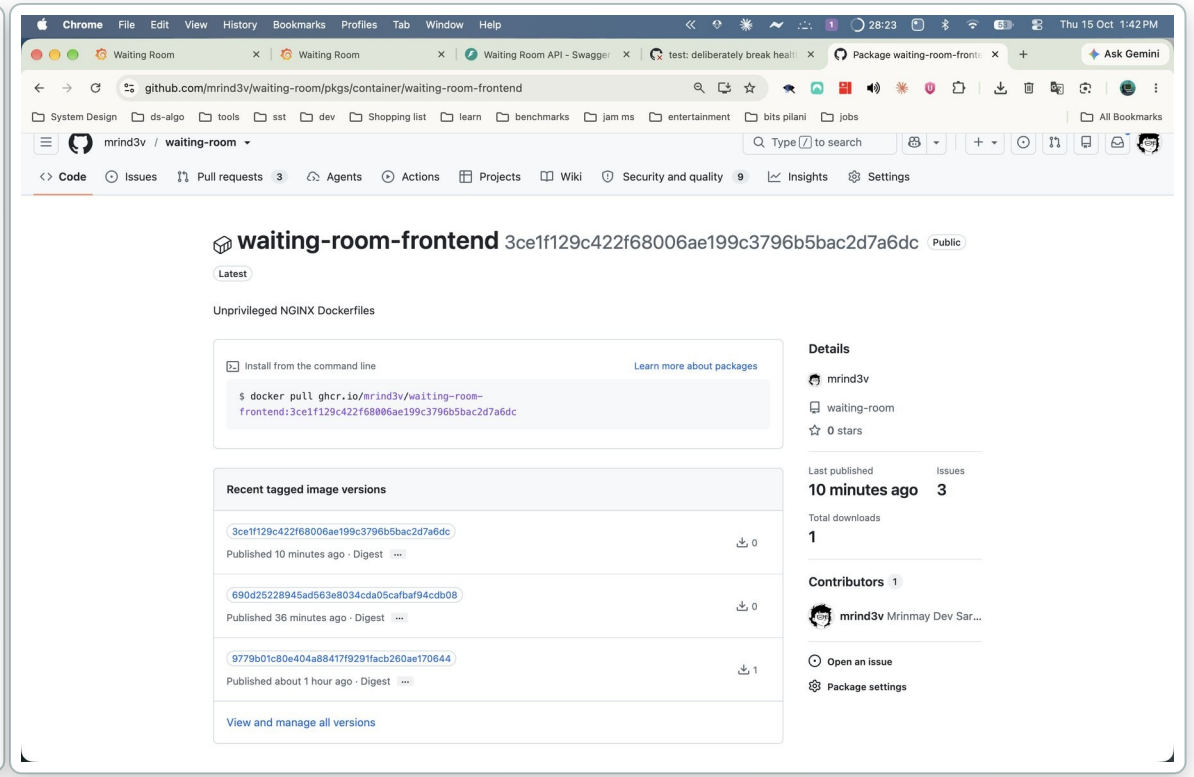


Red: one failing test blocks build, scan and deploy

Images ship to GHCR tagged by commit SHA



waiting-room-backend



waiting-room-frontend

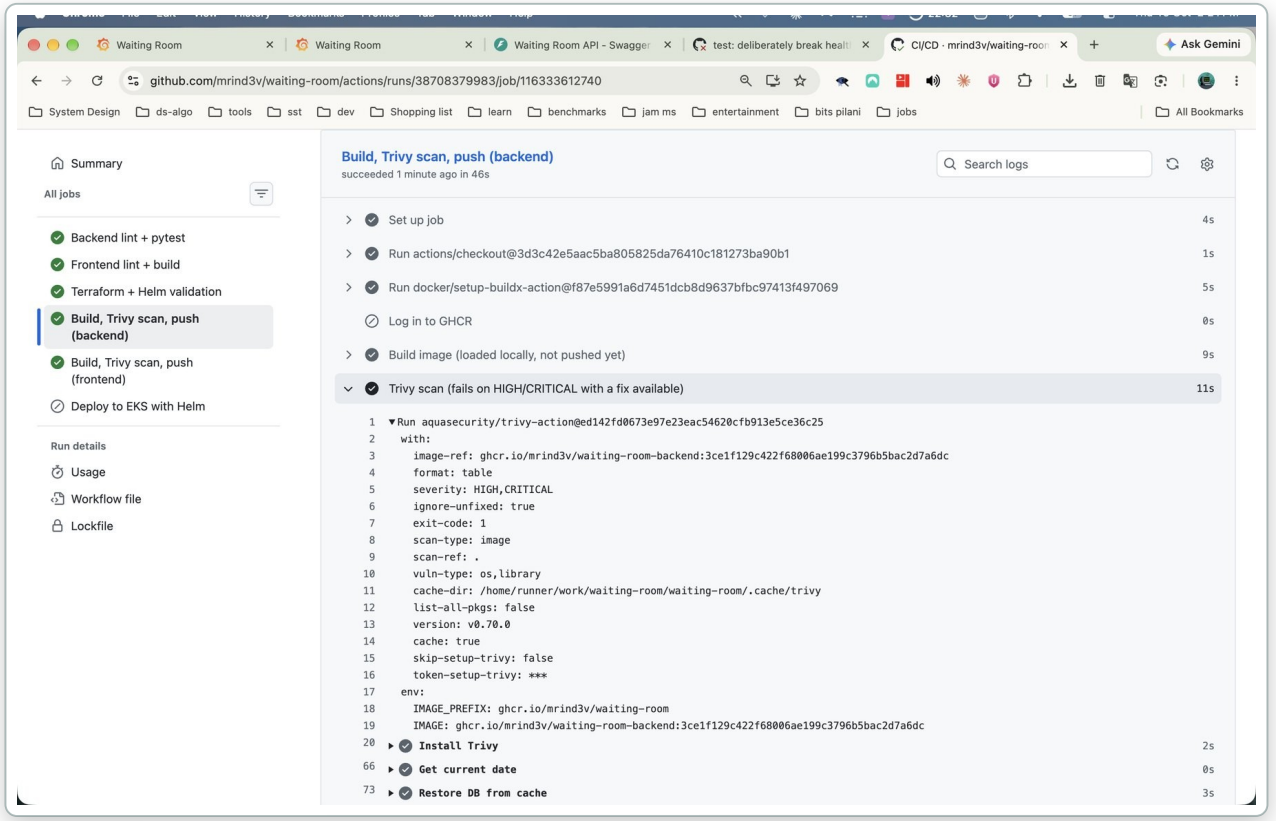
Trivy gates every image

Scans OS packages and dependencies in both images

Fails the build on fixable HIGH or CRITICAL CVEs

Latest result: 0 findings

Publishes a SARIF report to the Security tab



Trivy report from the pipeline run

Security runs through the whole pipeline

Non-root everywhere

Containers drop all capabilities and use read-only root filesystems

Actions pinned by SHA

A 2026 supply-chain attack hit a mutable scanner tag

No secrets in Git

Passwords are generated in-cluster; .env stays local

Dependabot weekly

Pinned versions and base images stay current

Short-lived CI identity

An IAM user that only exists during the AWS window

CORS closed by default

Same-origin through the proxy; no wildcard

04

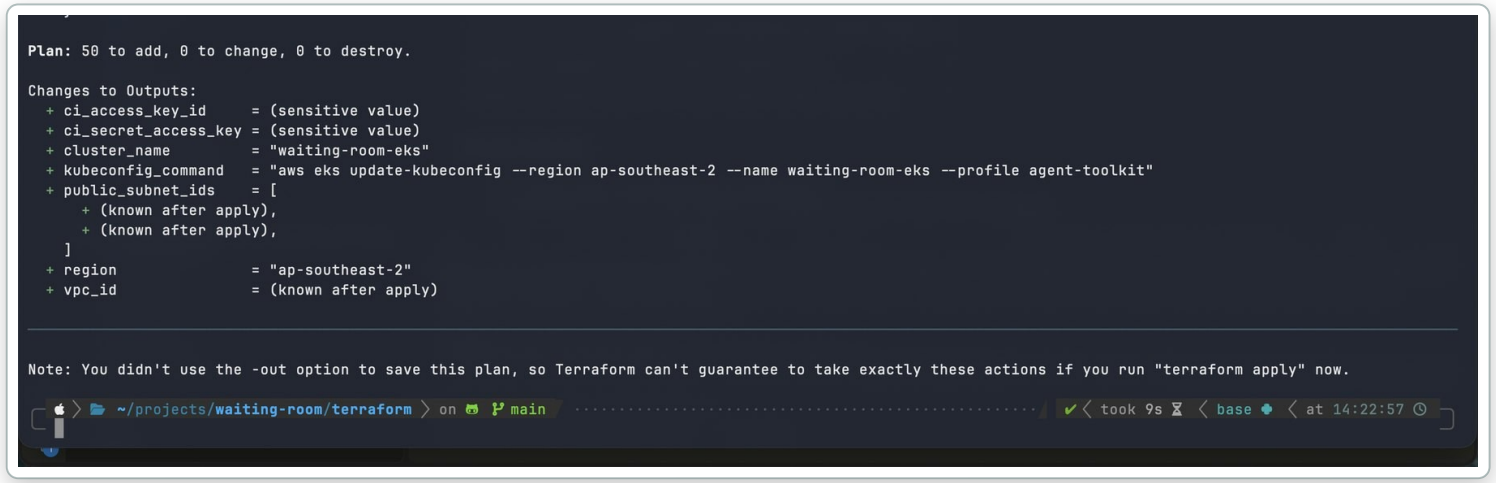
Infrastructure and Kubernetes

Terraform builds AWS. Helm runs the app. Autoscaling and monitoring keep it healthy

The Free plan shaped every AWS decision

Constraint	My decision
Spot instances are blocked	One on-demand m7i-flex.large node
Only a few instance types are allowed	m7i-flex.large: 2 vCPU, 8 GiB
No IAM OIDC provider	EBS CSI uses EKS Pod Identity; CI uses a short-lived IAM user
Every dollar counts	Public subnets, no NAT gateway, no RDS, no KMS key
Old versions cost more	Kubernetes 1.35 on standard support

Terraform builds the whole stack from code



terraform plan: 50 to add, 0 to change, 0 to destroy



terraform apply: complete in 12 minutes

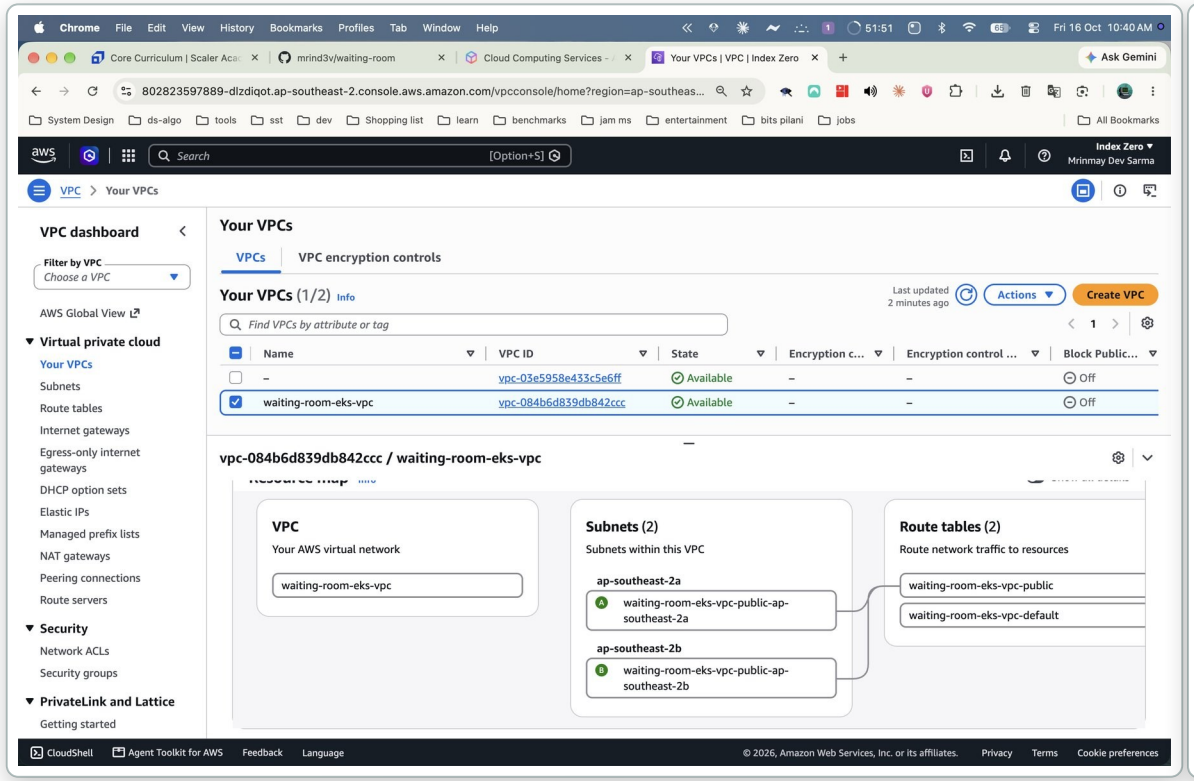
50

resources planned and applied

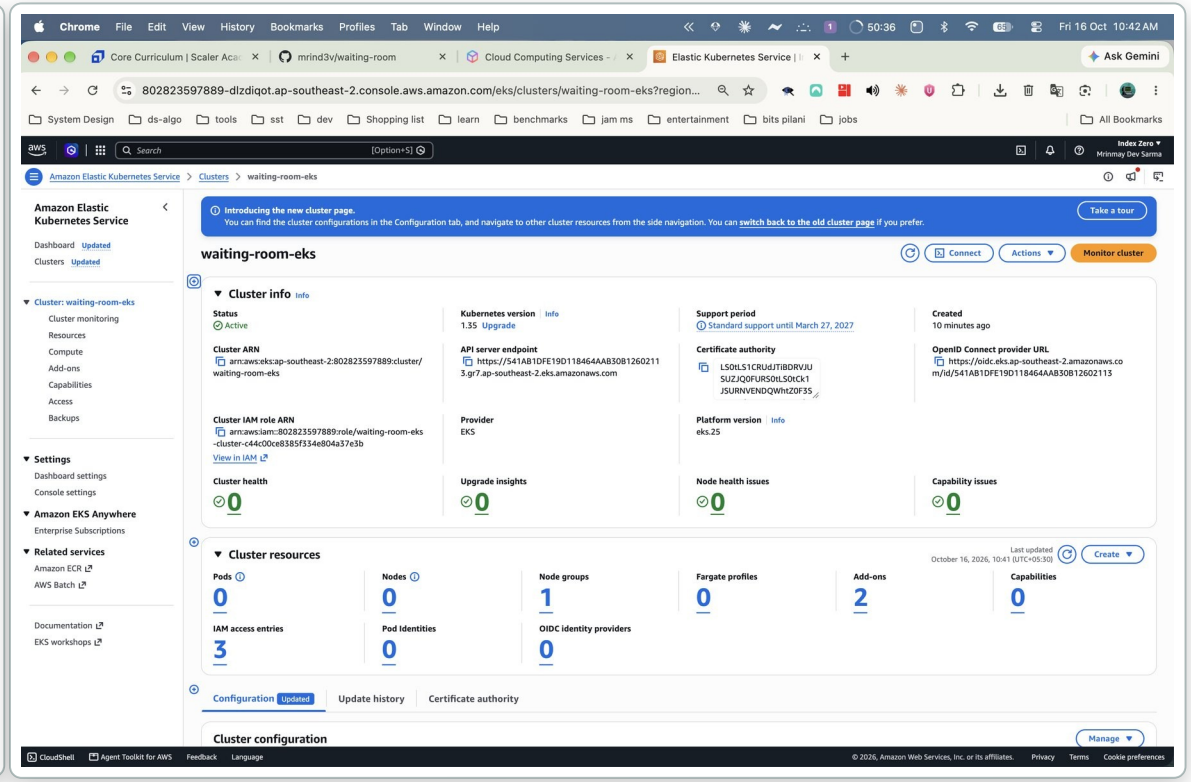
12 min

to create the EKS cluster

AWS Console confirms the VPC and EKS cluster



VPC with two public subnets



EKS 1.35 cluster, status Active

Helm packages the app for Kubernetes

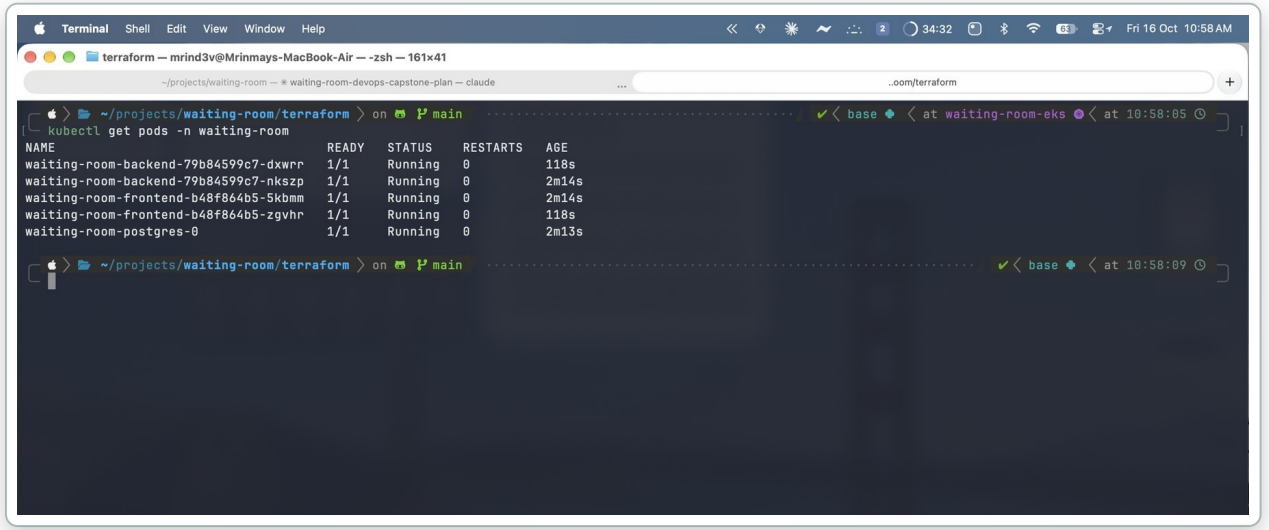
Deployments: 2 backend and 2 frontend pods

StatefulSet: PostgreSQL with a gp3 volume

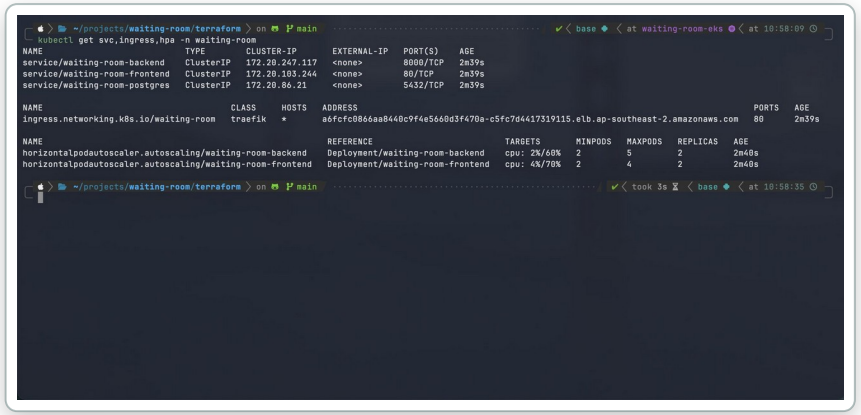
ClusterIP services plus one Ingress

Hardened pods: read-only filesystems, no capabilities

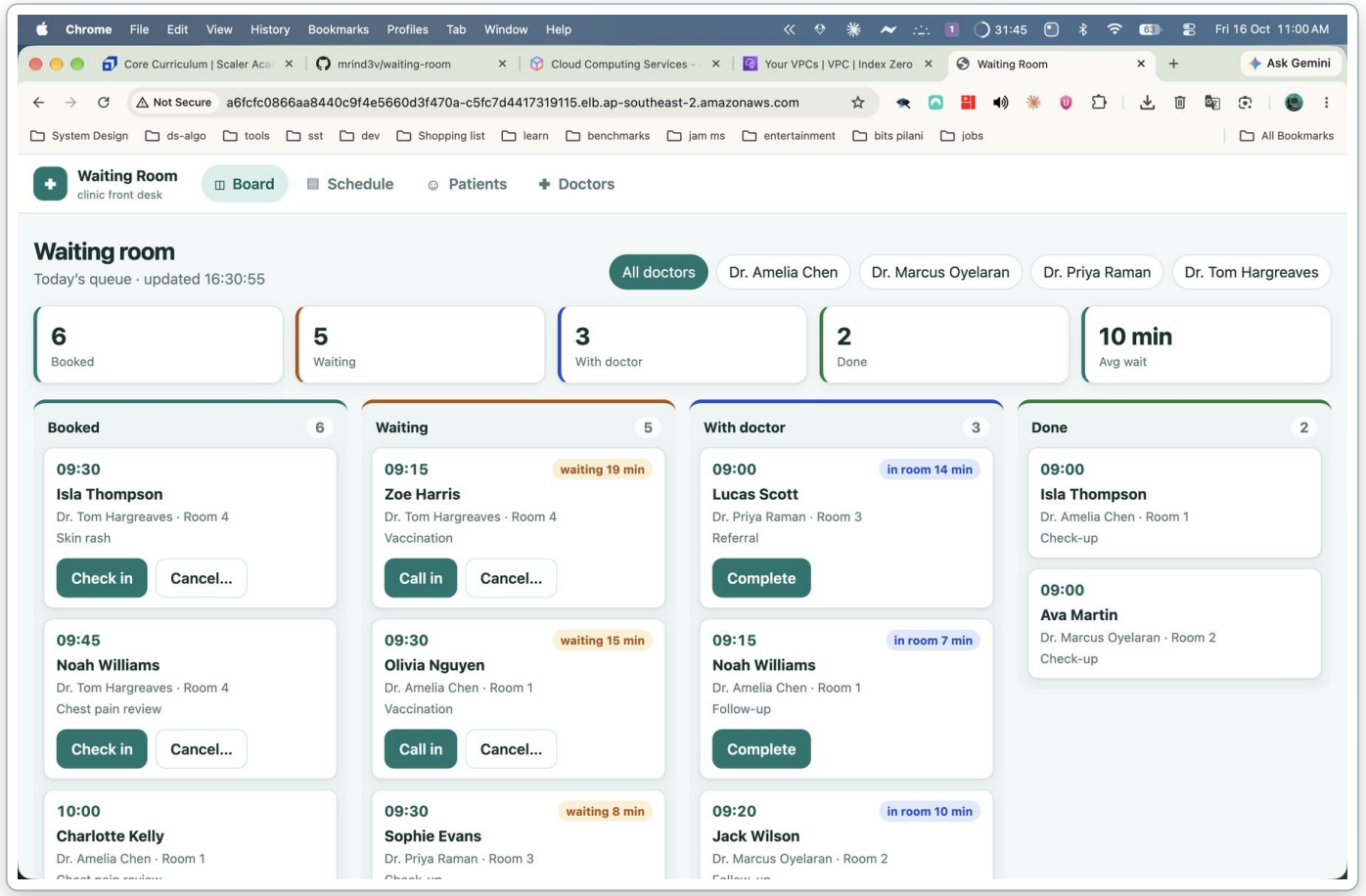
Autoscaling: CPU-based HPAs



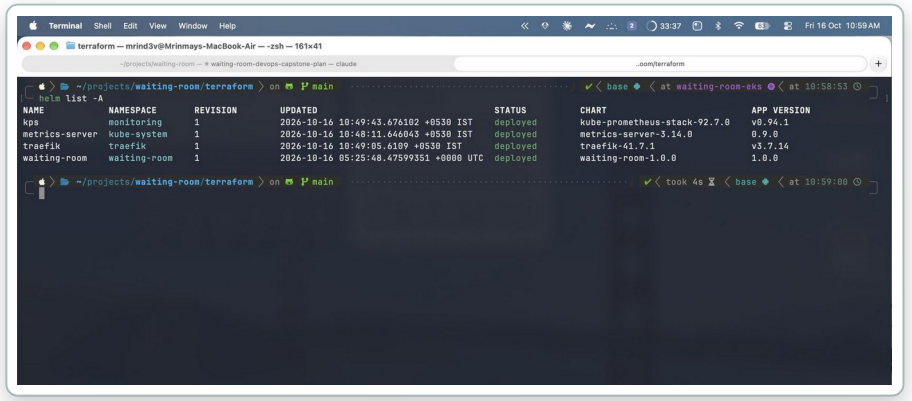
kubectl get pods: everything Running



The app runs on EKS behind one load balancer



Live on the NLB hostname

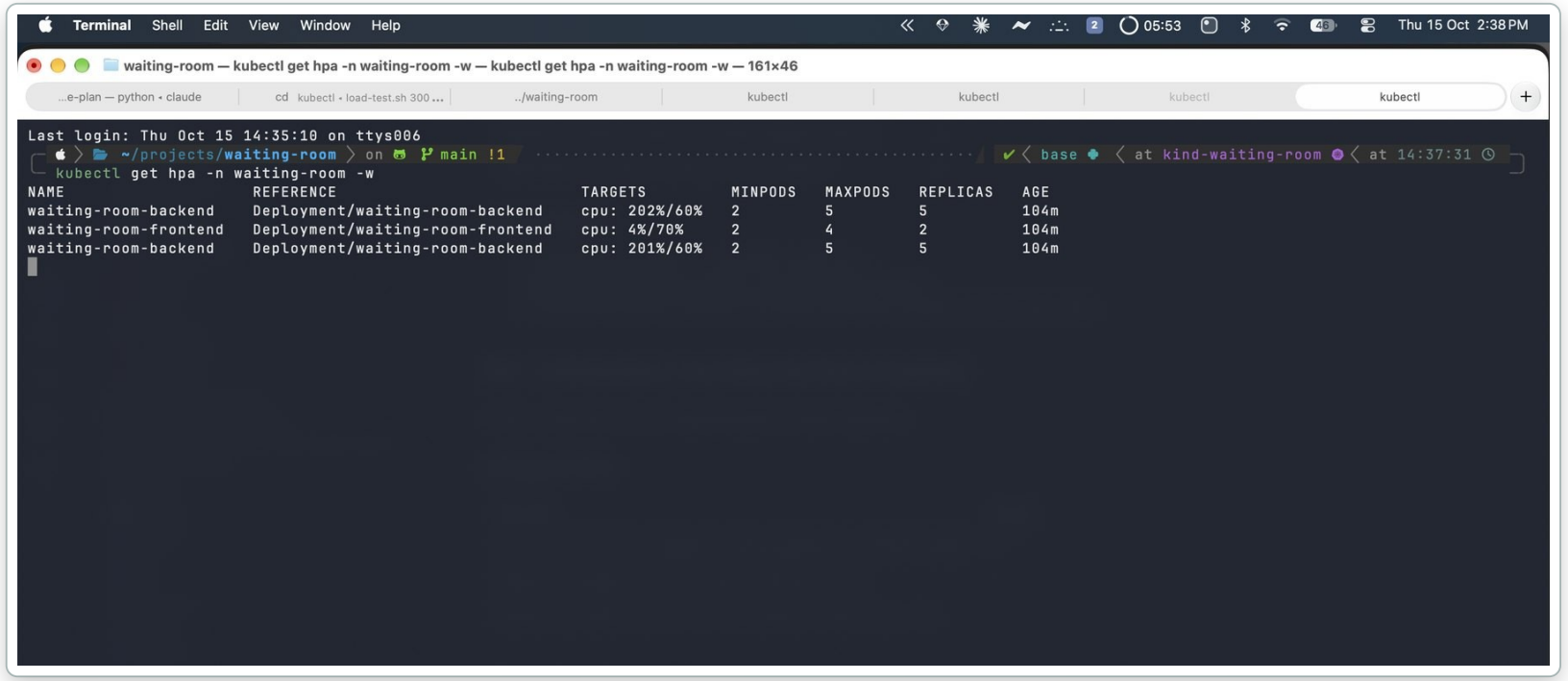


One NLB is the only public entry point

Traefik routes /api to FastAPI and / to nginx

Three Helm releases run the app, ingress and monitoring

The autoscaler adds pods under load



HPA under load (kind rehearsal)

2 → 5

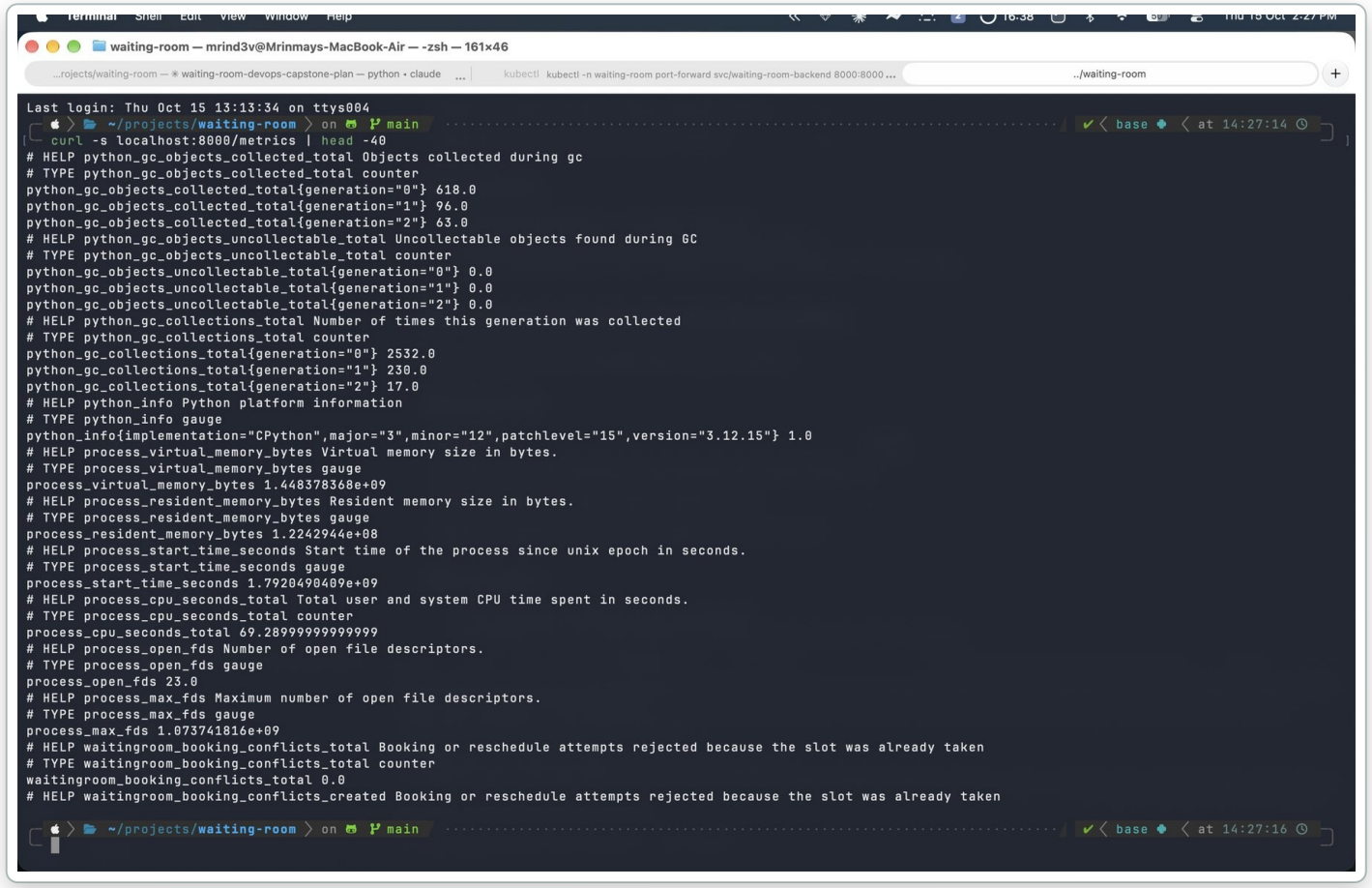
backend pods under load

0

errors in 58,785 requests

Target: 60% CPU · min 2 pods · max 5 pods

The backend exposes Prometheus metrics



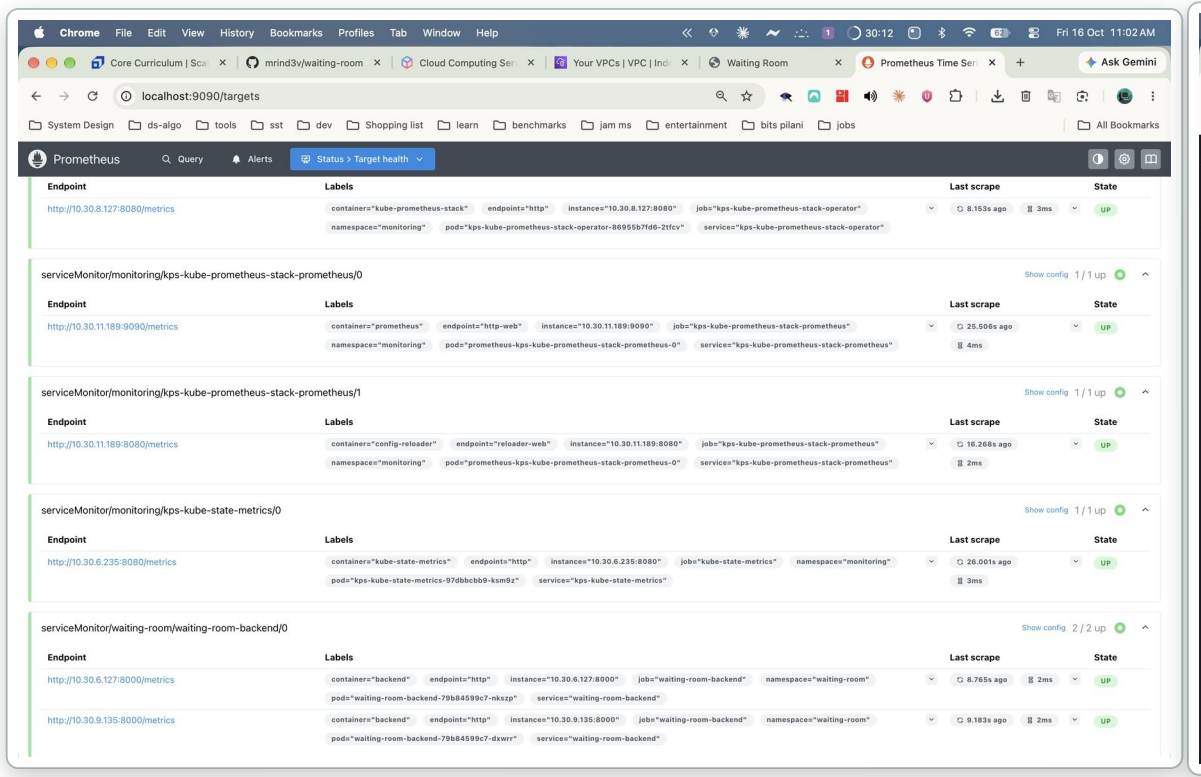
HTTP metrics: requests, latency and status codes

Bookings counted as they happen

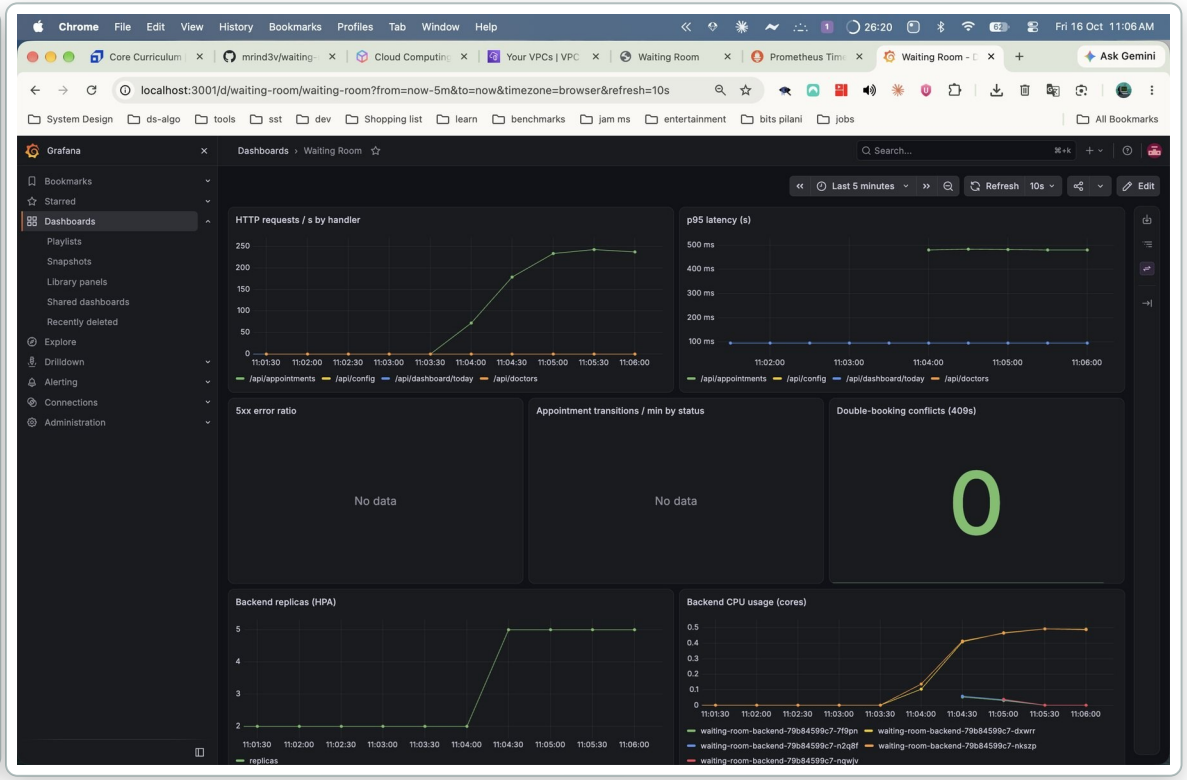
Conflicts: every 409 increments a counter

Transitions: counted per status

Prometheus scrapes it and Grafana shows it



Prometheus: backend targets UP



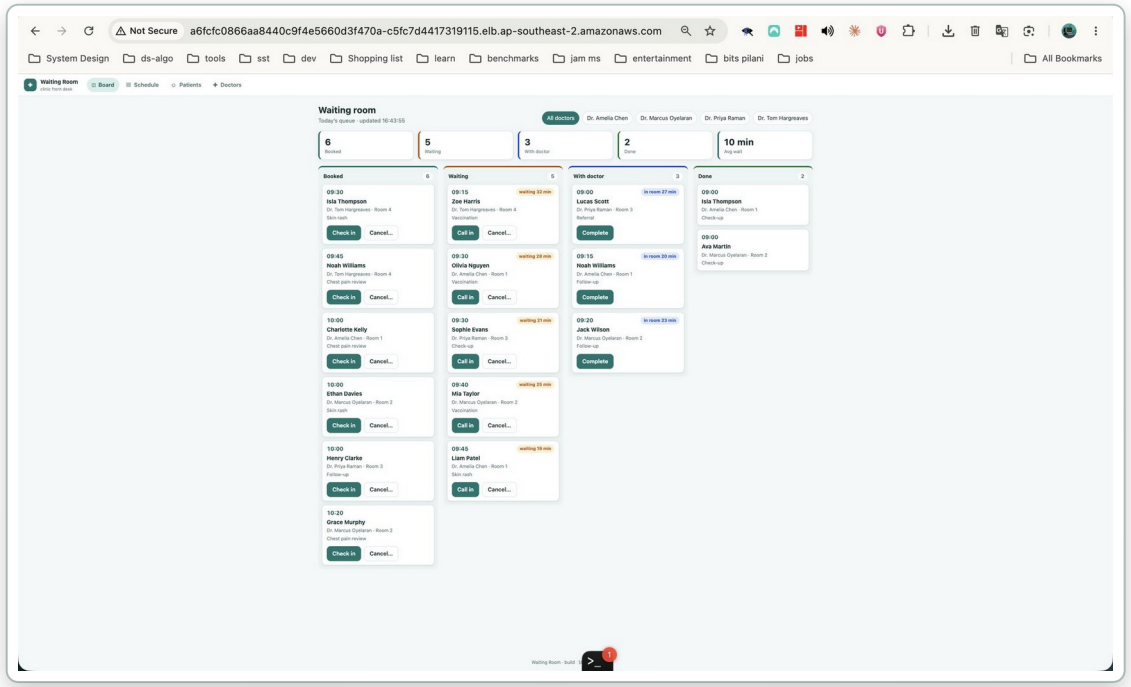
Grafana: the Waiting Room dashboard on EKS

05

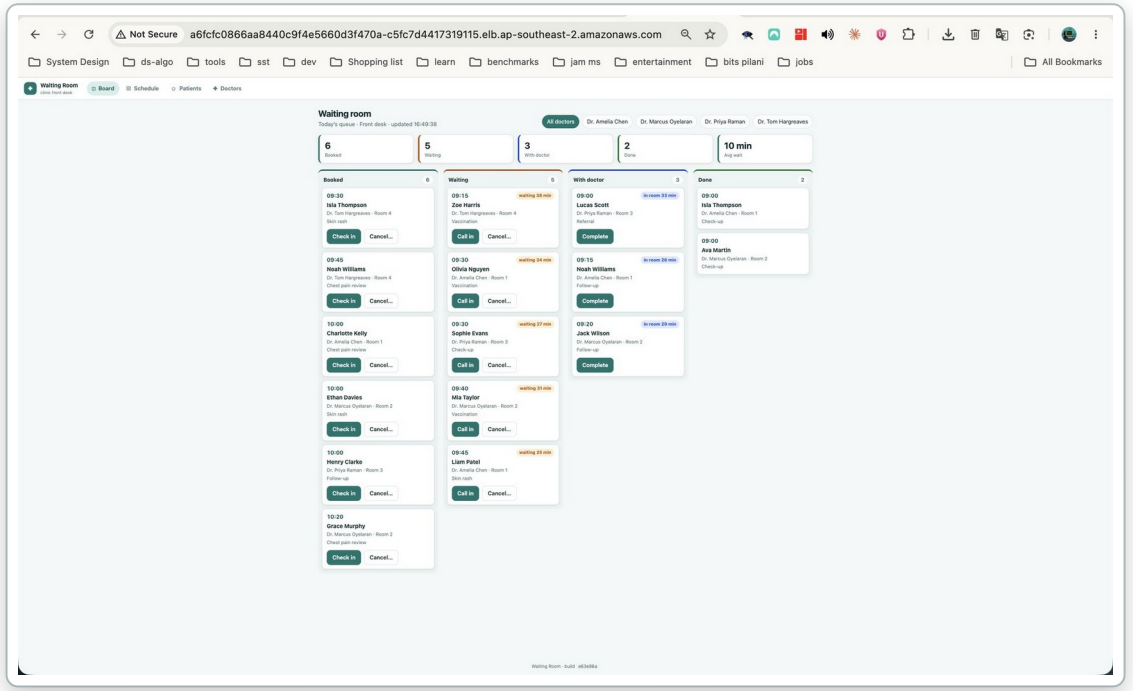
Operate and learn

Ship a change. Diagnose failures. Control cost. Reflect honestly

One commit became a live deployment

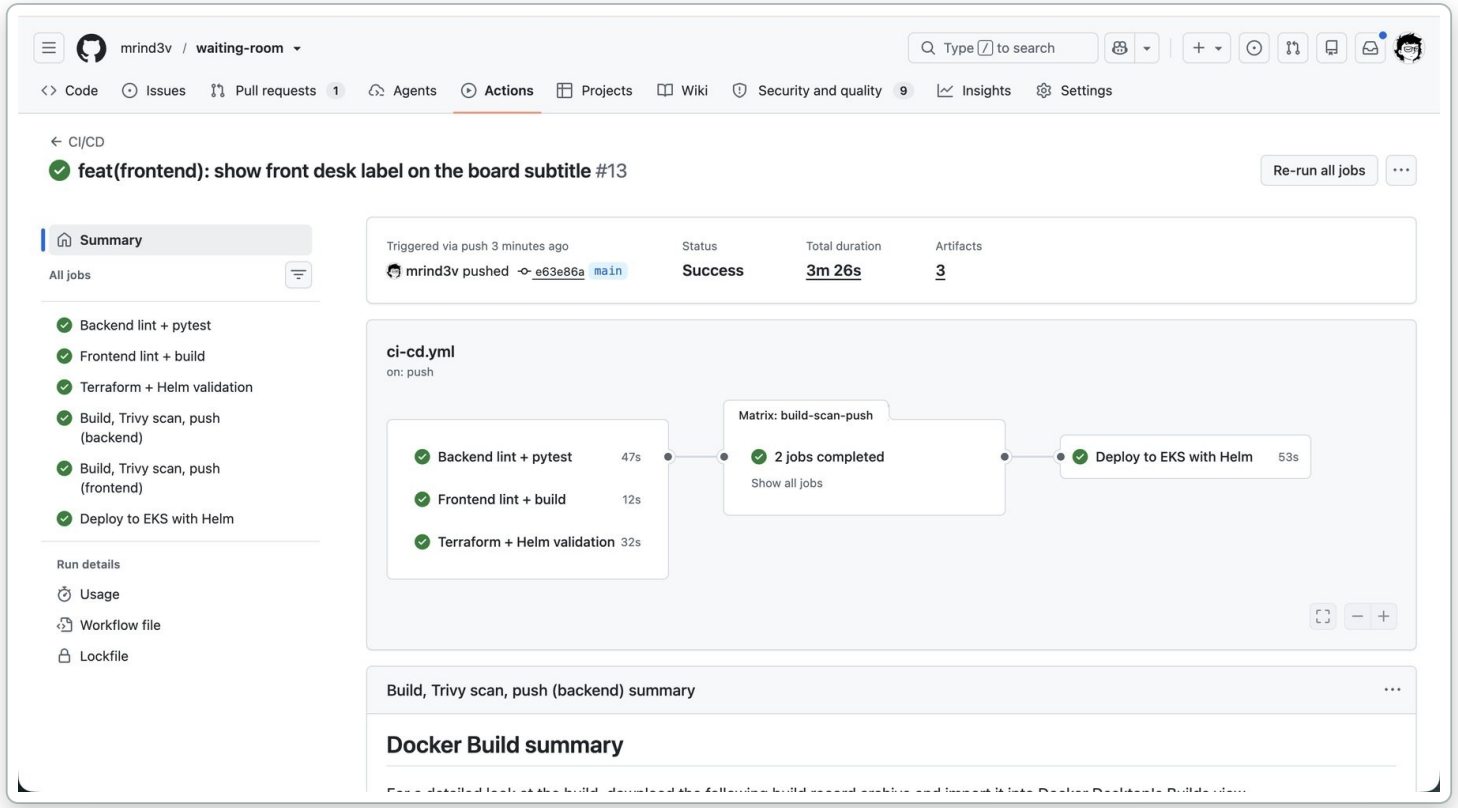


Before: build 10a6c28



After: build e63e86a, with the new subtitle

The pipeline ships the change in minutes



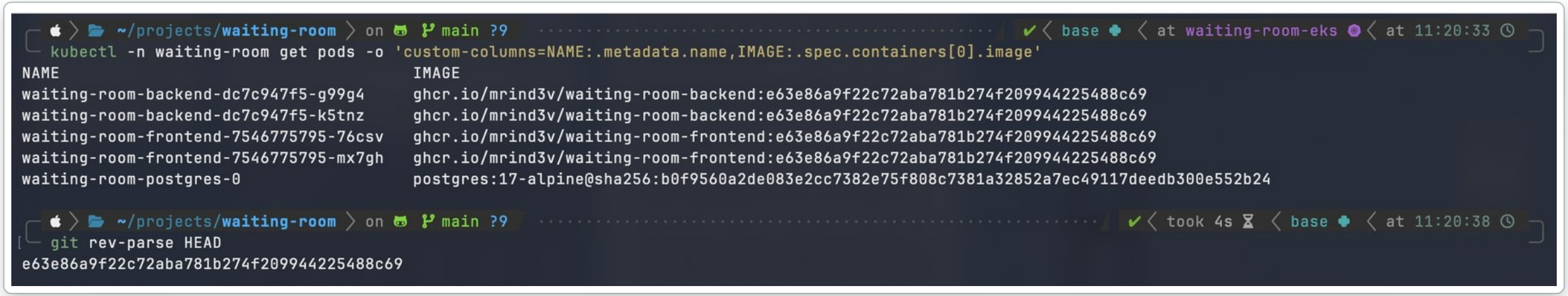
All six jobs pass, including the Helm deploy

Tested, scanned and built in about three minutes

Pushed both images to GHCR, tagged with the new SHA

Deployed with a rolling, atomic Helm upgrade

The cluster now runs the new image



Pod images match the commit SHA printed by git rev-parse HEAD

Commit to pod

The image tag equals the full Git SHA

Easy rollback

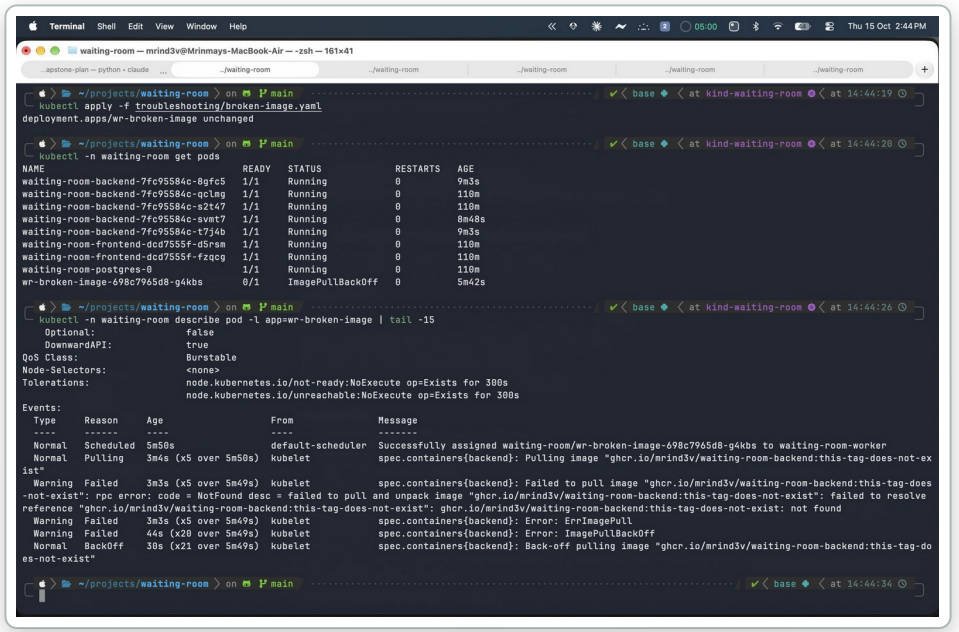
Deploy an earlier SHA to go back

Atomic upgrade

A failed rollout reverts automatically

A troubleshooting lab trains the diagnosis

Symptom	Cause	Fix
ImagePullBackOff	Image tag does not exist	Set the right tag
Service has no traffic	Selector matches no pods	Patch the selector
Running, not Ready	Database host is wrong	Correct DB_HOST



ImagePullBackOff reproduced and diagnosed

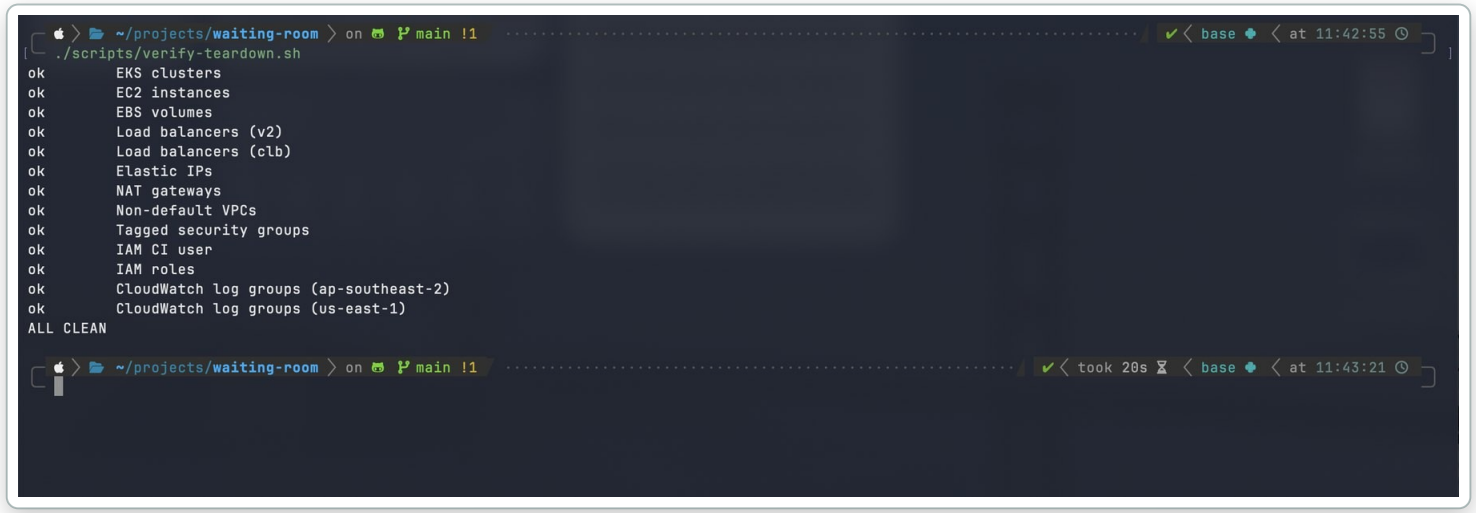
Events first. Labels connect Services to pods. Liveness is not readiness.

AWS cost stayed near zero, nothing runs now

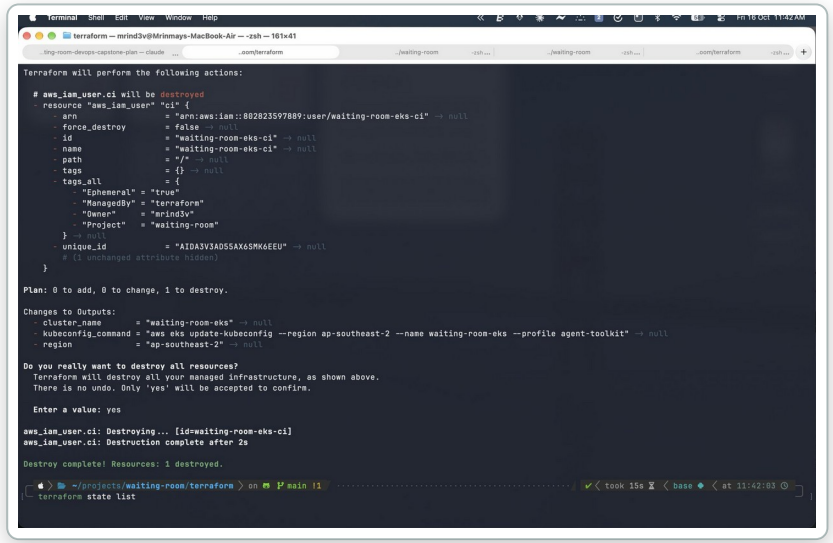
$0.26
per hour while running

6
budget alerts guard the account

ALL CLEAN
after teardown verification



verify-teardown.sh: nothing left in either region



terraform destroy

Five problems that taught me the most

Grafana crashed repeatedly	Out-of-memory kills at 256 Mi. I raised the limit.
Docker build failed	email-validator was missing from requirements. I pinned it.
Bootstrap script failed on macOS	An empty array broke under set -u. I fixed the expansion.
Terraform destroy failed	force_destroy calls an API the Free plan blocks. I turned it off.
The load balancer URL timed out	The browser tried HTTPS; only HTTP is exposed. I used http://.

Where Waiting Room goes beyond the reference

Reference	Waiting Room
CORS allows every origin	CORS closed by default
create_all next to Alembic	Alembic migrations only
nginx runs as root	nginx-unprivileged, non-root
NAT gateway, 2 nodes, Kubernetes 1.31	No NAT, 1 node, Kubernetes 1.35
Passwords in values and compose	Secrets generated in the cluster
Actions pinned by tag	Actions pinned by commit SHA
Tests on SQLite	Tests on real PostgreSQL

Every grading module has evidence

Module	Evidence
M1 Application	16 endpoints, React UI, Alembic migrations, /health
M2 Testing	36 pytest tests, 97% coverage, a test database
M3 Git and GitHub	Public repo with meaningful commits and a complete .gitignore
M4 Docker	Multi-stage non-root images; compose starts three services
M5 CI/CD	Six-job pipeline; GHCR images tagged by SHA
M6 DevSecOps	Trivy on both images, fails on HIGH and CRITICAL
M7 Terraform	VPC, EKS and a node group: plan, apply, destroy
M8 Kubernetes + Helm	Chart, 2+ replicas, ClusterIP, Ingress, running on EKS
M9 Observability	/metrics, Prometheus targets UP, Grafana dashboard
M10 Documentation	README, runbooks and this presentation

THANK YOU

What I would build next

- HTTPS with a domain and ACM certificates
- Managed PostgreSQL on RDS
- GitHub OIDC instead of an IAM user
- Alertmanager rules for failed bookings

github.com/mrind3v/waiting-room

Mrinmay Dev Sarma · DevOps · Instructor: Nensi Ravaliya